

ECOM EXPERTS

Default With Stock

Manual de usuario e instalación

defaultwithstock · 05/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Pestaña «General»

Pestaña «Resolución de problemas»

Sobre la multitienda

¿Guarda bien?

5. Cómo se usa, día a día

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Qué se ha revisado

Qué se ha arreglado en esta revisión

Fallo introducido durante la revisión y corregido

Qué se ha comprobado en una tienda de verdad

Qué queda pendiente

9. Ficha técnica

Módulo: defaultwithstock · **Versión:** 1.1.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0

1. Qué hace y para quién

Cuando un producto tiene combinaciones (tallas, colores, capacidades...), PrestaShop marca una de ellas como «predeterminada». Es la que sale ya elegida en cuanto alguien abre la ficha, la que decide qué foto y qué precio se ven, y la que aparece en los listados de categoría y en el buscador.

El problema llega cuando esa combinación predeterminada se queda sin existencias. PrestaShop **no la cambia solo**: la ficha sigue abriéndose por la talla agotada. El cliente entra, ve «Agotado» o el botón de comprar deshabilitado, y se va, aunque las otras seis tallas del mismo producto estén disponibles. En un catálogo con cientos de referencias esto pasa todos los días y nadie se entera hasta que se miran las ventas.

Este módulo se encarga de eso. Se queda escuchando los movimientos de existencias de la tienda —una venta, una devolución, una importación, un ajuste a mano— y, en cuanto uno de ellos deja a cero la combinación predeterminada de un producto, pasa la marca de «predeterminada» a otra combinación del mismo producto que sí tenga existencias. El cliente que entre a partir de ese momento se encuentra la ficha lista para comprar.

Es un módulo silencioso: no añade ninguna pantalla a la tienda, no toca el diseño, no manda correos y no tiene mantenimiento. Se instala, se deja encendido y trabaja solo. Está pensado para tiendas de moda, calzado, cosmética o electrónica: cualquier catálogo con muchas combinaciones por producto y existencias que se mueven a diario.

2. Compatibilidad

PrestaShop	1.7, 8.x y 9.x (declara <code>min 1.7</code>)
Probado en vivo	PrestaShop 8.2.7 (instalado, configurado y ejecutado; ver sección 8)
PHP	7.4, 8.1, 8.2, 8.3 y 8.4 (comprobado con <code>php -1</code> en 7.4, 8.1 y 8.3)
Multitienda	Sí. Las opciones son globales ; el cálculo y la escritura de la combinación predeterminada son por tienda
Idiomas	Castellano y ficha inglesa de origen. Sistema de traducción nuevo (<code>.xlf</code>)
Dependencias	Ninguna. Sin Composer, sin librerías externas, sin tablas propias
Base de datos	No crea ninguna tabla

3. Instalación

1. Sube la carpeta `defaultwithstock` a `/modules/` de tu tienda, o instala el ZIP desde **Módulos** → **Gestor de módulos** → **Subir un módulo**.
2. Pulsa **Instalar**.
3. Entra en **Configurar** para revisar las opciones (la sección 4 explica cada una).

Qué crea al instalarse:

- Se engancha al aviso `actionUpdateQuantity` del núcleo, que es por donde PrestaShop comunica cualquier cambio de existencias.
- Crea tres opciones de configuración: `DEFAULTWITHSTOCK_LIVE_MODE`, `DEFAULTWITHSTOCK_SKIP_BACKORDER` y `DEFAULTWITHSTOCK_DEBUG`. Al instalar quedan en **encendido**, **encendido** y **apagado** respectivamente.
- Crea la carpeta `logs/` dentro del módulo, protegida con su `index.php` y un `.htaccess` que impide servirla y ejecutarla por web.
- **No crea tablas, ni pestañas en el menú, ni controladores, ni tareas programadas.**

Al desinstalar se borran las tres opciones y las del registro en la herramienta de versiones, y se suelta el aviso. **No se toca el catálogo:** las combinaciones predeterminadas se quedan tal y como estuvieran en ese momento. Desinstalar no deshace los cambios ya hechos, sencillamente deja de hacer cambios nuevos.

4. Configuración

La pantalla está en **Módulos** → **Gestor de módulos** → **Cambia la combinación por defecto con stock** → **Configurar**. Es un único formulario con dos pestañas y un solo botón de guardar.

Pestaña «General»

Configuración

GENERAL RESOLUCIÓN DE PROBLEMAS

Activar ☒ SI

Cuando un movimiento de stock deja a cero la combinación por defecto de un producto, la marca de «por defecto» pasa a la combinación con stock de número mas bajo. Desactivalo para dejar el catalogo intacto sin tener que desinstalar el modulo.

Saltar los productos que se pueden pedir sin stock ☒ SI

Recomendado. Si un producto admite pedidos estando agotado (reserva), la combinacion vacia se sigue pudiendo vender y cambiar la predeterminada solo despista al comprador. Desactivalo unicamente si quieres que el cambio se haga en todos los productos, admitan reserva o no.

Guardar

Activar. El interruptor general. Con él en «Sí» el módulo reacciona a los movimientos de existencias; con él en «No» no toca absolutamente nada del catálogo, aunque siga instalado.

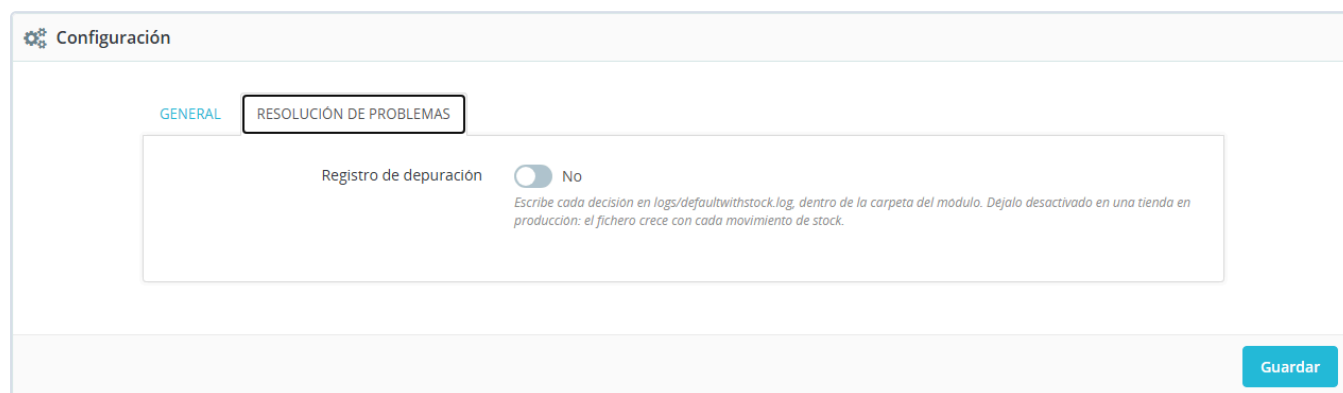
Sirve para dos cosas muy concretas. La primera, probar: lo enciendes, miras un par de días cómo se comportan las fichas, y si no te convence lo apagas sin tener que desinstalar ni perder la configuración. La segunda, las temporadas raras: si vas a hacer una importación masiva de existencias o un recuento de inventario en el que muchos productos van a pasar por cero de forma temporal, apágalo mientras dure y vuelve a encenderlo al terminar. Así evitas que las fichas vayan cambiando de talla predeterminada durante el recuento.

Saltar los productos que se pueden pedir sin stock. Déjalo en «Sí» salvo que tengas un motivo claro para lo contrario.

PrestaShop permite marcar productos como «se puede pedir aunque no haya existencias» (venta bajo reserva o *backorder*), bien producto a producto, bien de forma general en **Parámetros de la tienda** → **Productos**. En esos productos, que una combinación esté a cero **no** significa que no se pueda comprar: el cliente la pide igual y se la sirves cuando llegue. Cambiarle la combinación predeterminada en ese caso no arregla nada y sí despista, porque le estás apartando de la talla que había elegido la tienda por defecto y que sigue siendo vendible.

Con esta opción en «Sí», el módulo comprueba antes de actuar si el producto admite pedidos estando agotado y, si los admite, lo deja en paz. Con ella en «No», el cambio se hace en todos los productos por igual, admitan reserva o no.

Pestaña «Resolución de problemas»



Registro de depuración. Apagado en una tienda en producción. Encendido, el módulo escribe una línea en `modules/defaultwithstock/logs/defaultwithstock.log` cada vez que le llega un movimiento de existencias y cada vez que cambia una combinación predeterminada:

```
[2026-09-04 08:46:49] hook: producto 1 combinacion 1 tienda 1 cantidad 0
[2026-09-04 08:46:49] producto 1 tienda 1: default 1 (stock 0) -> 2
```

Enciéndelo sólo cuando quieras entender por qué un producto concreto ha cambiado (o por qué no ha cambiado), mira el fichero y vuelve a apagarlo. **Ojo: crece con cada movimiento de existencias de toda la tienda**, así que en una tienda con ventas se hace grande en horas. La carpeta `logs/` no se puede abrir desde el navegador.

Sobre la multitienda

Si tienes varias tiendas, la pantalla te avisa arriba: **las tres opciones de esta pantalla son globales**, valen para todas las tiendas a la vez. Lo que sí es por tienda es el trabajo de verdad: cuando se mueven existencias en la tienda A, el módulo mira las existencias de la tienda A y escribe la combinación predeterminada de la tienda A. La tienda B no se toca. Si las tiendas comparten existencias (`share_stock` del grupo), se respeta esa configuración del núcleo.

¿Guarda bien?

Sí, comprobado releendo la base de datos después de guardar, tanto al encender una opción como al apagarla (sección 8).

5. Cómo se usa, día a día

No se usa. Ésa es la gracia. Una vez encendido, no hay ninguna pantalla que visitar, ni un botón que pulsar, ni una tarea programada que vigilar. El módulo actúa solo, en el mismo momento en que se mueven las existencias.

Actúa siempre que PrestaShop registre un cambio de existencias, venga de donde venga:

- una **venta** en la tienda que deja la talla a cero;
- una **devolución** o una cancelación que devuelve unidades;
- una **importación** de existencias por CSV;
- un cambio **a mano** desde la ficha del producto en el back-office;
- una **edición de pedido** desde el back-office;
- el **webservice** o cualquier módulo de sincronización con un ERP.

Lo que hace, exactamente, cada vez que le llega uno de esos avisos:

1. Mira si el aviso es de una combinación (los productos sin combinaciones no le interesan).
2. Si está activada la opción de reserva, comprueba si ese producto se puede vender agotado. Si se puede, no hace nada.
3. Vuelve a leer de la base de datos **todas** las combinaciones de ese producto en esa tienda, con sus existencias reales de ese momento.
4. Si la combinación predeterminada sigue teniendo existencias, no hace nada.
5. Si está a cero (o el producto no tenía ninguna marcada), pone como predeterminada la primera combinación con existencias, entendiendo por «primera» la de identificador más bajo, que es el orden en que se crearon.

Dos detalles importantes que conviene saber:

- **Revisa el producto entero, no sólo la combinación que se ha movido.** Si la talla M se agotó ayer por otra vía y hoy se vende una unidad de la talla L, al procesar el movimiento de la L el módulo se da cuenta de que la predeterminada (la M) está a cero y lo arregla. Se va reparando solo con el uso.

- **No devuelve la marca a su sitio cuando la combinación original repone.** Si la talla S era la predeterminada, se agotó y la marca pasó a la M, cuando vuelvas a tener existencias de la S la marca se queda en la M. Es a propósito: el módulo sabe cuál está agotada, pero no puede saber cuál preferías tú. Si quieres volver a la S, cámbialo a mano en la ficha del producto.
-

6. Qué ve el cliente en la tienda

Nada nuevo. El módulo no añade ningún bloque, ningún aviso ni ningún texto a la tienda: lo único que cambia es **cuál de las combinaciones sale ya elegida** al abrir la ficha.

Estas dos capturas son el mismo producto, la misma ficha y el mismo cliente. En la primera, la combinación predeterminada es *Talla S / Color Blanco* y tiene existencias:

-20%

NUEVO



HUMMINGBIRD PRINTED T-SHIRT

28,92 €

23,14 € 20% DE DESCUENTO

Impuestos incluidos

Regular fit, round neckline, short sleeves. Made of extra long staple pima cotton.

Tamaño: S

S ▼

Color: Blanco



Additional information

Texto grabado (+4,24 €)

Cantidad

1



AÑADIR AL CARRITO



Compartir



! Este producto no admite devolución

Este producto no admite devolución ni desistimiento por tratarse de un artículo personalizado o excluido por ley (art. 103 TRLGDCU). Revise bien su elección antes de comprar.

Este artículo se fabrica en lotes de 8 unidades

Puedes llevarte 7 unidades más con un 50% de descuento

Este artículo se fabrica de 8 en 8 unidades. Como has pedido 1, en el taller sobrarían 7 unidades que acabarían en la basura.

SI ACEPTAS:

	AHORA	SI ACEPTAS
Unidades que recibes	1	8 (+7)
Precio del artículo	23,14 €	104,11 € (+80,97 €)
Te sale a	23,14 € / ud	13,01 € / ud

Si aceptas, añadimos esas 7 unidades a tu pedido: recibirás 8 en lugar de 1 y pagarás 80,97 € más (11,57 € por unidad en vez de 23,14 €). Tu pedido pasa de 23,14 € a 104,11 €.

Te salen a 13,01 € por unidad en lugar de 23,14 €.

Las unidades extra se fabrican expresamente para este pedido y no admiten devolución por desistimiento. Si no aceptas, recibirás las 1 unidades que pediste.

➔ Al aumentar las unidades puede variar el coste de envío.

☐ **Sí, quiero las 7 unidades extra por 80,97 € más. He leído las condiciones.**

Si no marcas la casilla, recibirás solo las unidades que has pedido.

Este producto no suma puntos por estar rebajado.

Security policy
(edit with the Customer Reassurance module)

Delivery policy
(edit with the Customer Reassurance module)

Return policy
(edit with the Customer Reassurance module)

Descripción

Detalles del producto

Symbol of lightness and delicacy, the hummingbird evokes curiosity and joy. Studio Design' PolyFaune collection features classic products with colorful patterns, inspired by the traditional japanese origamis. To wear with a chino or jeans. The sublimation textile printing process provides an exceptional color rendering and a color, guaranteed overtime.

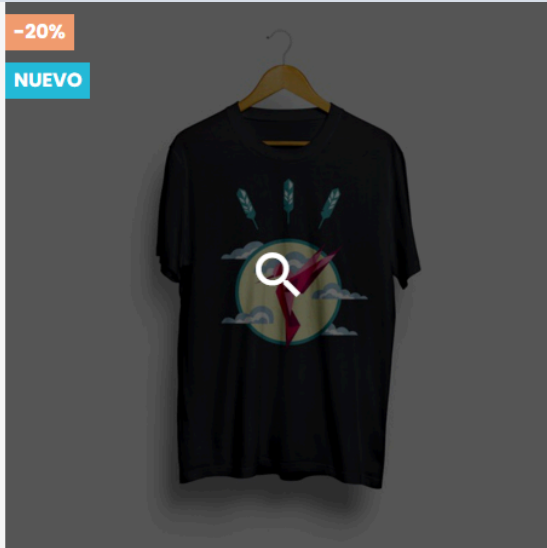
Comentarios (0)

No hay reseñas de clientes en este momento.

Se deja la talla S blanca a cero. Sin el módulo, la ficha seguiría abriéndose por esa combinación y el cliente se encontraría el producto sin poder comprarlo. Con el módulo, la ficha pasa sola a *Talla S / Color Negro*, que sí tiene existencias: cambia la foto, cambia el color marcado y el botón **Añadir al carrito** sigue disponible.

-20%

NUEVO



HUMMINGBIRD PRINTED T-SHIRT

28,92 €

23,14 € 20% DE DESCUENTO

Impuestos incluidos

Regular fit, round neckline, short sleeves. Made of extra long staple pima cotton.

Tamaño: S

S ▼

Color: Negro



Additional information

Texto grabado (+4,24 €)

Cantidad

1



AÑADIR AL CARRITO



Compartir



! Este producto no admite devolución

Este producto no admite devolución ni desistimiento por tratarse de un artículo personalizado o excluido por ley (art. 103 TRLGDCU). Revise bien su elección antes de comprar.

Este artículo se fabrica en lotes de 8 unidades

Puedes llevarte 7 unidades más con un 50% de descuento

Este artículo se fabrica de 8 en 8 unidades. Como has pedido 1, en el taller sobrarían 7 unidades que acabarían en la basura.

SI ACEPTAS:

	AHORA	SI ACEPTAS
Unidades que recibes	1	8 (+7)
Precio del artículo	23,14 €	104,11 € (+80,97 €)
Te sale a	23,14 € / ud	13,01 € / ud

Si aceptas, añadimos esas 7 unidades a tu pedido: recibirás 8 en lugar de 1 y pagarás 80,97 € más (11,57 € por unidad en vez de 23,14 €). Tu pedido pasa de 23,14 € a 104,11 €.

Te salen a 13,01 € por unidad en lugar de 23,14 €.

Las unidades extra se fabrican expresamente para este pedido y no admiten devolución por desistimiento. Si no aceptas, recibirás las 1 unidades que pediste.

➔ Al aumentar las unidades puede variar el coste de envío.

☐ **Sí, quiero las 7 unidades extra por 80,97 € más. He leído las condiciones.**

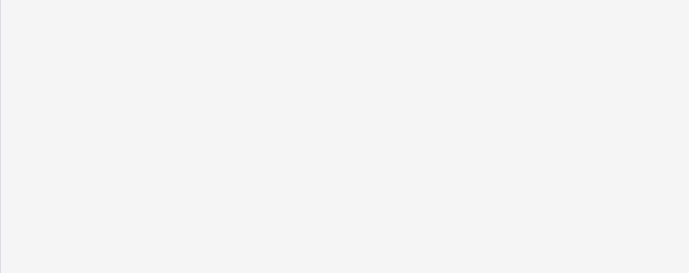
Si no marcas la casilla, recibirás solo las unidades que has pedido.

Este producto no suma puntos por estar rebajado.

Security policy
(edit with the Customer Reassurance module)

Delivery policy
(edit with the Customer Reassurance module)

Return policy
(edit with the Customer Reassurance module)

Descripción	Detalles del producto
	Symbol of lightness and delicacy, the hummingbird evokes curiosity and joy. Studio Design' PolyFaune collection features classic products with colorful patterns, inspired by the traditional japanese origamis. To wear with a chino or jeans. The sublimation textile printing process provides an exceptional color rendering and a color, guaranteed overtime.
 Comentarios (0)	
No hay reseñas de clientes en este momento.	

El efecto se nota igual en los listados de categoría, en el buscador y en los bloques de productos, porque todos ellos usan la combinación predeterminada para decidir la foto y el precio que enseñan.

7. Preguntas frecuentes y problemas

He agotado una talla y la ficha no ha cambiado. Repasa, por este orden: que el interruptor **Activar** esté en «Sí»; que el producto **no** admita pedidos sin existencias (si los admite y tienes la segunda opción en «Sí», el módulo lo salta a propósito); y que quede alguna otra combinación con existencias, porque si están todas a cero no hay a dónde mover la marca y el módulo no toca nada. Si aun así no cuadra, enciende el registro de depuración, repite el movimiento y mira el fichero de registro.

¿Puede dejar un producto sin combinación predeterminada? No. Sólo escribe cuando ha encontrado una combinación con existencias a la que pasar la marca. Si no encuentra ninguna, se queda como estaba.

¿Y si todas las combinaciones están agotadas? No hace nada. La ficha se queda con la que tuviera, y el producto se comporta como cualquier producto agotado de tu tienda.

¿Ralentiza la tienda o las importaciones? No de forma apreciable. Cada movimiento de existencias le cuesta **una sola consulta** a la base de datos, más una escritura únicamente en el caso —poco frecuente— de que haya que cambiar la marca. Una importación de 5.000 líneas son 5.000 consultas ligeras.

¿Cambia la combinación predeterminada de mis otras tiendas? No. Lee y escribe sólo sobre la tienda en la que se ha movido el stock.

Vendo bajo reserva y no quiero que cambie nada. Deja **Saltar los productos que se pueden pedir sin stock** en «Sí». Si además quieres pararlo del todo, pon **Activar** en «No».

Quiero que vuelva a la talla original cuando reponga. No lo hace, y no está previsto. Ver el segundo detalle de la sección 5.

¿Puedo elegir yo el orden de preferencia? En esta versión no: siempre elige la combinación con existencias de identificador más bajo, es decir, la más antigua de las que creaste. Queda anotado como mejora pendiente en la sección 8.

He desinstalado el módulo. ¿Se deshacen los cambios? No. Los cambios ya hechos se quedan; simplemente dejan de hacerse cambios nuevos.

8. Seguridad

Qué se ha revisado

Se han leído los ficheros del módulo (11 en total, 6 de ellos PHP) buscando inyección de SQL, XSS, fallos de CSRF y de autorización, subida de ficheros, salto de directorios, ejecución de código, secretos escritos en el código, fugas de información, llamadas de red inseguras, recorridos masivos sobre datos históricos y compatibilidad con PHP 8.

No se ha encontrado ningún problema de seguridad, ni antes ni después de los cambios. El módulo no tiene controladores, ni pantallas en la tienda, ni llamadas AJAX, ni sube ficheros, y no recibe ningún dato del usuario: sus únicos datos de entrada son los que le pasa el propio núcleo de PrestaShop, y se convierten a entero antes de usarlos. Las consultas se construyen sólo con enteros ya convertidos.

Qué se ha arreglado en esta revisión

1. El módulo no llegaba a hacer nada en la mayoría de escenarios — `defaultwithstock.php`, método `hookActionUpdateQuantity`. Éste era el fallo grave, y sólo se ve ejecutando el módulo. El código construía la lista de combinaciones con `Product::getAttributesResume()`, que por el camino pide a PrestaShop la precisión de redondeo de la moneda. En cualquier contexto donde la moneda no esté cargada —tareas programadas, importaciones por línea de órdenes, webservice, sincronizaciones con un ERP— eso lanza un `TypeError`. Y el módulo tenía un `catch` vacío que se lo tragaba sin dejar rastro: ni error en pantalla, ni línea en el registro, ni aviso. Simplemente no cambiaba la combinación y nadie se enteraba. Comprobado reproduciéndolo:

```
TypeError: ComputingPrecision::getPrecision(): Argument #1 ($displayPrecision)
must be of type int, null given, called in classes/Context.php on line 557
```

Ahora el módulo no instancia el producto ni usa `getAttributesResume()` para leer: hace una consulta directa. El problema desaparece de raíz.

2. Las excepciones ya no se tragan en silencio — mismo fichero. El `catch` vacío se ha sustituido por un registro real: todo fallo se apunta en el registro de PrestaShop con `PrestaShopLogger::addLog` y, si está encendida la depuración, también en el fichero del módulo. El módulo sigue sin poder tumbar una venta o una importación, pero ahora deja rastro.

3. Decidía con una foto de las existencias anterior a la escritura — mismo fichero. El código leía las existencias con `StockAvailable::getQuantityAvailableByProduct()`, que sirve desde una memoria intermedia de la propia petición. El núcleo lanza el aviso `actionUpdateQuantity` antes de limpiar esa memoria (comprobado en el código de 8.2.7 y de 9.1.1), así que en las rutas donde ya se había consultado esa misma existencia en la misma petición, el módulo recibía el valor **anterior** al cambio y decidía con datos caducados. Ahora se relee el estado real de la base de datos, contra la conexión principal, después de la

escritura. En las rutas que se han podido probar en vivo el núcleo acaba limpiando esa memoria por otra vía antes del aviso, así que no se ha llegado a reproducir un fallo visible por esta causa concreta; el cambio es preventivo y, además, es el que hace posible la mejora de rendimiento del punto 5.

4. En multitienda decidía con una tienda y escribía en todas — mismo fichero. Leía las existencias de la tienda del movimiento, pero la marca de «predeterminada» se leía y se escribía con el contexto de tienda que hubiera en ese momento. Cuando ese contexto es «todas las tiendas» —el caso del importador, del webservice y de las tareas programadas— las funciones del núcleo escriben en **todas** las tiendas a la vez. Es decir: un movimiento en la tienda A podía cambiar la combinación predeterminada de la tienda B, donde esa combinación sí tenía existencias. Ahora la lectura y la escritura se hacen dentro del contexto de la tienda del movimiento, y ese contexto se restaura siempre al salir.

5. Cada movimiento de existencias costaba unas $2N+2$ consultas — mismo fichero. Se cargaba el producto entero en todos los idiomas y se pedía el resumen de combinaciones, que por dentro hace una consulta de existencias por combinación; y después el módulo repetía otra consulta de existencias por combinación. Para un producto de 30 combinaciones eran unas 60 consultas por cada línea de pedido o fila de importación. Ahora es **una sola consulta**, que además devuelve ya la marca de predeterminada y las existencias juntas.

6. Cambiaba la combinación también en productos que se venden agotados — mismo fichero. Se ha añadido la comprobación `Product::isAvailableWhenOutOfStock()`, la misma que usa el núcleo, y la opción de pantalla para desactivarla.

7. No había forma de apagarlo, y la opción que existía no se leía nunca — mismo fichero. La clave `DEFAULTWITHSTOCK_LIVE_MODE` se creaba al instalar y no se consultaba en ningún sitio; tampoco había pantalla de configuración. Ahora hay una pantalla con las tres opciones y la clave se respeta de verdad.

8. La memoria interna del núcleo se quedaba desactualizada tras el cambio — mismo fichero. `setDefaultAttribute()` actualiza las tablas pero no la memoria de `Product`. En la misma petición, cualquier cosa que preguntara por la combinación predeterminada seguía recibiendo la antigua. Se reinicia con `Product::getDefaultAttribute($id, 0, true)` y se limpia la memoria de precios.

9. Estructura y datos de identificación — todos los ficheros. Se ha eliminado la carpeta `upgrade/`, prohibida por las normas del proyecto y que además sólo contenía el esqueleto vacío del generador para una versión que nunca se habría ejecutado. Se ha borrado `stockpr.log`, un resto de depuración de 1 byte que viajaba en el paquete del cliente, y se ha sacado `Readme.md` fuera del módulo. Las cabeceras de todos los ficheros, que eran las de PrestaShop SA 2007-2024, son ahora las de Ecom Experts 2026 con licencia AFL-3.0, y el autor declarado ha pasado de `ecomyseo` a `Ecom Experts`. Se ha añadido `addnewfeatures()` para las migraciones. Se ha creado la carpeta `logs/` con su `index.php` y su `.htaccess`.

10. El módulo salía sin traducir — `defaultwithstock.php` y `translations/`. Usaba el sistema antiguo (`$this->l()`) con las cadenas de origen escritas en castellano y sin ningún fichero de traducción, así que en una tienda en otro idioma aparecía siempre en castellano y con el nombre a medias. Ahora declara el sistema nuevo, las cadenas de origen están en inglés y hay traducción al castellano en `translations/es-ES/ModulesDefaultwithstockAdmin.es-ES.xlf`. Las 19 cadenas del código están comprobadas una a una contra el fichero de traducción con un guion automático.

11. La comprobación de versión se repetía en cada visita — mismo fichero. Se ha añadido el registro del módulo en la herramienta de versiones, con verificación de certificado activada y tres segundos de espera como máximo. Si el servicio no contesta, ahora se anota el intento y no se reintenta hasta pasada una hora, para no dejar al usuario esperando tres segundos cada vez que abre la pantalla.

Fallo introducido durante la revisión y corregido

Merece la pena dejarlo escrito porque es exactamente la clase de fallo que ninguna lectura del código detecta. Al reescribir el método se sustituyó el bloque `finally` por código colocado **después** del `try`. En PHP, un `return` dentro del `try` se salta ese código. El resultado: el cerrojo que evita la reentrada se quedaba cerrado para siempre.

Y no fallaba de vez en cuando, fallaba **siempre**: al guardar existencias, el núcleo lanza el aviso primero para el producto completo (con la combinación a `0`) y sólo después para la combinación de verdad. El primer aviso salía por el `return` de «esto no es una combinación», dejaba el cerrojo cerrado, y el segundo —el que importa— se lo encontraba cerrado y no hacía nada. `php -1` seguía limpio y no había ningún error en ninguna parte. Se detectó al ejecutar el módulo contra la tienda y se corrigió devolviendo el `finally`.

Qué se ha comprobado en una tienda de verdad

Todo lo anterior se ha verificado en una instalación real de **PrestaShop 8.2.7** con los datos de demostración, no sólo leyendo el código:

- **Instalación:** correcta, con el aviso registrado (`FALLAN: 0`).
- **Pruebas de regresión** (15 comprobaciones, ejecutadas dentro de PrestaShop): que el cambio se hace aunque la memoria intermedia esté cargada con el valor antiguo; que dos movimientos seguidos en la misma petición se procesan los dos; que no se toca nada si la predeterminada conserva existencias; que queda exactamente una combinación marcada y las dos copias de `cache_default_attribute` quedan sincronizadas; que con el módulo apagado no se toca nada; y que los productos con venta bajo reserva se respetan. **15 de 15 correctas.**
- **Las pruebas se han validado reintroduciendo los fallos:** al volver a meter el fallo del cerrojo, 5 comprobaciones fallan; al restaurarlo, vuelven a pasar. Una prueba que no salta cuando se repone el fallo no vale de nada.
- **Pantallas, con navegador** (24 comprobaciones): que la configuración abre sin error, que sale traducida, que las dos pestañas cuelgan del formulario, que se pulsa cada interruptor de cada pestaña, que al guardar se guarda de verdad —releyendo la base de datos— tanto al encender como al apagar, y **cero errores de JavaScript** en todas las pantallas.
- **La tienda:** se ha dejado a cero la combinación predeterminada de un producto real y se ha comprobado con el navegador que la ficha pasa a ofrecer otra combinación con existencias y que el botón de comprar sigue disponible. Son las dos capturas de la sección 6.
- **Prueba de humo** del back-office: sin problemas atribuibles a este módulo.

Qué queda pendiente

Nada de esto es un fallo; son mejoras que cambiarían el comportamiento y por eso no se han hecho sin pedirlo:

- **Elegir el criterio de preferencia.** Hoy es siempre «la combinación con existencias de identificador más bajo». Podría ser la más barata, la de más existencias, o un orden que fije el comerciante.
- **Devolver la marca a la combinación original cuando repone.** Requiere guardar cuál era la predeterminada «de verdad» antes del primer cambio.
- **Comprobación en una instalación multitienda real.** El comportamiento por tienda está implementado y razonado sobre el código del núcleo de 8.2.7 y 9.1.1, pero la tienda de pruebas tiene una sola tienda, así que no se ha podido ejecutar con varias.
- **Comprobación en vivo sobre PrestaShop 9.x.** Se ha verificado que todo lo que usa el módulo existe igual en 9.1.1, pero las pruebas se han ejecutado sobre 8.2.7.

9. Ficha técnica

Estructura de ficheros (11 ficheros, 69 KB):

```
defaultwithstock/
  defaultwithstock.php           clase principal, sin namespace
  index.php
  logs/
    .htaccess                   impide servir y ejecutar la carpeta
    index.php
  translations/
    index.php
    es-ES/
      index.php
      ModulesDefaultwithstockAdmin.es-ES.xlf
  views/
    index.php
  templates/
    index.php
  admin/
    index.php
    gmartos_update.tpl          aviso de versión nueva
```

Sin `src/`, sin `config/`, sin `composer.json`, sin `vendor/` y sin `upgrade/`: estructura clásica, como el resto de módulos de la casa.

Aviso al que se engancha

Aviso	Método	Qué hace
<code>actionUpdateQuantity</code>	<code>hookActionUpdateQuantity(array \$params)</code>	Todo el trabajo del módulo

Del aviso usa `id_product`, `id_product_attribute` e `id_shop`; todos se convierten a entero.

Opciones de configuración (todas globales, `Configuration::getGlobalValue`)

Clave	Por defecto	Qué es
<code>DEFAULTWITHSTOCK_LIVE_MODE</code>	1	Interruptor general
<code>DEFAULTWITHSTOCK_SKIP_BACKORDER</code>	1	Saltar productos vendibles sin existencias
<code>DEFAULTWITHSTOCK_DEBUG</code>	0	Registro a fichero
<code>GMARTOS_LAST_PING_defaultwithstock</code>	0	Fecha del último registro correcto de versión
<code>GMARTOS_LAST_TRY_defaultwithstock</code>	—	Fecha del último intento (freno de una hora)
<code>GMARTOS_UPDATE_AVAILABLE_defaultwithstock</code>	—	Si hay versión nueva
<code>GMARTOS_LATEST_VERSION_defaultwithstock</code>	—	Cuál es esa versión

Tablas que lee (ninguna propia, ninguna que cree)

`product_attribute_shop` y `stock_available` para leer, en una sola consulta filtrada por producto y tienda, con la restricción de tienda del núcleo (`StockAvailable::addSqlShopRestriction`) para respetar las existencias compartidas entre tiendas de un grupo.

Tablas que escribe (a través de las funciones del núcleo, nunca con SQL propio)

`product_attribute` y `product_attribute_shop` (columna `default_on`), y `product` y `product_shop` (columna `cache_default_attribute`), mediante `Product::deleteDefaultAttributes()` y `Product::setDefaultAttribute()`.

Otros

Controladores	Ninguno
Pestañas de menú	Ninguna
Tareas programadas	Ninguna
Llamadas AJAX	Ninguna
Ficheros de registro	<code>modules/defaultwithstock/logs/defaultwithstock.log</code> , sólo con la depuración encendida
Registro de errores	Registro de PrestaShop (Parámetros avanzados → Registros), gravedad 3
Salida a Internet	Sólo <code>https://modules.gmartos.es/api.php</code> , una vez por semana, con certificado verificado y 3 s de espera máxima
Dominio de traducción	<code>Modules.Defaultwithstock.Admin</code>
Permisos de carpeta	<code>logs/</code> con permiso de escritura para el usuario del servidor web